

大数据分析算法实验课方案

第七、八周实验设计

 课程名称：大数据分析算法（双语）

目录



01 实验背景

介绍实验的研究背景与意义



02 实验方案（基础版）

基础实验流程与操作规范



03 实验方案（提高版）

进阶技术参数与优化策略



04 实验方案（科研版）

创新性研究方法与探索



05 实验报告要求对比

各版本报告的评分标准差异



06 总结与建议

实验总结与未来改进方向

实验背景

理论课对应章节

第七、八章（图数据分析、推荐系统）

实验目的

巩固理论知识，提升实践能力，培养创新思维

课程目标支撑

目标3（掌握典型算法）、目标4（应用现代工具）

核心知识领域



图数据分析

构建复杂网络模型，挖掘数据间的关联关系



推荐系统

基于协同过滤与深度学习，实现个性化推荐



现代工具应用

熟练掌握主流机器学习框架与数据分析平台

第七周实验



实验目标

结合图表示学习与 PageRank 算法，深入探究节点分类问题，验证算法在异构网络中的有效性。



核心内容

- 构建引文网络
- Node2Vec 生成节点嵌入特征
- 结合结构特征进行分类评估



实施步骤

1. 数据预处理与网络构建
2. PageRank 节点重要性排序
3. 嵌入生成与分类模型训练
4. 指标计算与可视化分析



实验过程掌握

- 3个概念：引文网络、图表示学习、节点分类
- 1个数据集：cora数据集。
- 4个技术：pagerank、node2vec、特征拼接、交叉验证。

1. 引文网络是什么，长什么样？

引文网络 = 论文之间互相引用形成的图

节点 (Node) : 一篇论文

边 (Edge) : 论文 A 引用了论文 B, 或者被 B 引用

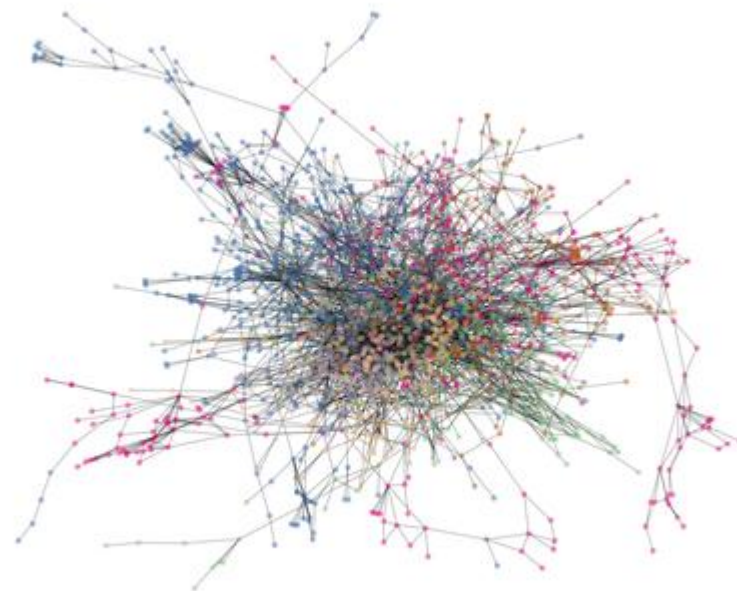
标签 (Label) : 每篇论文属于哪个学科

特征: 论文的标题 / 摘要单词向量

一句话总结

引文网络就是用“图”来表示“谁引用了谁”，节点是论文，边是引用关系。

同类论文高度聚集，形成明显的社区 / 簇
不同社区之间存在少量跨领域引用



2. 图表示学习是什么，能干什么？

图表示学习 = 把图里的节点变成计算机能理解的向量 (Embedding)

它在做什么？

让结构相似的节点，向量也相似。比如：经常互相引用的论文 → 向量靠得近

Node2Vec (图表示学习方法) 让机器读懂“谁和谁相连”，输出：每个节点一个 128 维向量

PageRank (图重要性算法) 衡量节点重要性，输出：每个节点一个 重要性分数

图表示学习能干嘛？

节点分类、推荐系统、药物发现、欺诈检测

一句话总结

图表示学习就是把“图结构”翻译成“向量”，让 AI 能读懂网络关系。

3. 什么是节点分类，可以怎么实现？

节点分类 = 给图里每个节点贴标签（分类）

在 Cora 里就是：

根据引用关系 → 判断这篇论文属于哪个学科

例子

- 论文 A 经常被“神经网络”领域的论文引用 → 分类为 Neural_Networks
- 论文 B 经常和“遗传算法”的论文相连 → 分类为 Genetic_Algorithms

节点分类怎么实现？

步骤 1：构建图，节点 = 论文，边 = 引用关系

步骤 2：提取特征

1.Node2Vec 学习结构特征（节点向量）

2.PageRank 计算重要性（重要性分数）

3.特征拼接

Node2Vec向量 + PageRank分数 = 最终特征

步骤 3：训练分类器用逻辑回归，输入：拼接好的特征，输出：论文类别

步骤 4：评估效果

- 交叉验证（看模型稳不稳定）

- 测试集评估（看真实准确率）

数据集: Cora 引文网络 (自动下载)

- 节点: 2708 篇学术论文
- 边: 论文引用关系
- 标签: 7 个主题类别
- 任务: 根据网络结构对论文进行分类

核心方法: 对比「纯 Node2Vec」/「Node2Vec+PageRank」两组实验

- 改变 Node2Vec 嵌入维度 (64/128/256) , 观察分类性能变化
- 改变 PageRank 阻尼因子 α (0.7 / 0.85 / 0.9) , 观察结果变化

1. PageRank

1. PageRank 是什么

- PageRank = 节点“重要性”分数
- 一篇论文被引用越多, PageRank 分数越高
- 输出: 每个节点 1 维数字 (表示重要程度)
- 作用: 告诉模型 这个点重要不重要
- 例子:
- 核心论文 → PageRank 高
- 边缘论文 → PageRank 低

2. Node2Vec

2. Node2Vec 是什么

- Node2Vec = 把节点变成向量 (embedding)
- 学习网络结构：谁和谁相连、邻居是谁
- 输出：每个节点 64/128 维向量
- 作用：表示节点的结构相似性
- 例子：
- 经常连在一起的节点 → 向量很像

3. 特征拼接

3. 特征拼接（特征连接）

- Node2Vec向量 + PageRank分数 = 最终特征
- Node2Vec: 结构信息 (128 维)
- PageRank: 重要性信息 (1 维)
- 拼起来变成:
- [节点向量 ...] + [重要性] = 更强的特征
- 为什么拼接? 结构 + 重要性 → 分类更准

4. 交叉验证

- 交叉验证 (交叉验证)
- 交叉验证 = 把训练集切5份, 轮流训练+验证
- 目的: 看模型稳不稳定, 避免运气好 / 运气差
- 流程 (5 折) :
- 训练集 → 切成 5 份
- 用 4 份训练, 1 份验证, 轮换 5 次, 取平均准确率
- 一句话: 交叉验证 = 选最好、最稳的模型

```
● (prvenv) 192:实验 lengyanmei$ python 2025pagerank.py
2026-04-19 11:04:12,315 - INFO - 开始加载Cora引文网络数据集...
2026-04-19 11:04:12,333 - INFO - 数据集统计: 节点数=2708, 边数=5278
2026-04-19 11:04:12,333 - INFO - 类别数=7, 特征维度=1433
2026-04-19 11:04:12,333 - INFO - 训练集数=140, 测试集数=1000
2026-04-19 11:04:12,333 - INFO - 开始计算PageRank, 阻尼因子alpha=0.85
2026-04-19 11:04:12,338 - INFO - PageRank计算完成, 分数范围=[0.000000, 1.000000]
2026-04-19 11:04:12,338 - INFO - 开始训练Node2Vec, 参数={'node2vec_dim': 128, 'node2vec_walk_len': 10, 'node2vec_num_walks': 20, 'node2vec_window': 5, 'pagerank_alpha': 0.85, 'lr_max_iter': 2000, 'cv_folds': 5, 'test_size': 0.2}
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
```

2026-04-19 11:04:14,929 - INFO - Node2Vec训练完成, 嵌入维度=128
2026-04-19 11:04:14,935 - INFO - 开始构造实验特征...
2026-04-19 11:04:14,936 - INFO - 特征构造完成: 纯Node2Vec维度=(2708, 128), 融合特征维度=(2708, 129)
2026-04-19 11:04:14,936 - INFO - 开始训练与评估: 纯Node2Vec
2026-04-19 11:04:22,313 - INFO - 纯Node2Vec - 5折交叉验证: Acc=0.2336, Macro-F1=0.1273
2026-04-19 11:04:23,303 - INFO - 纯Node2Vec - 测试集分类报告:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.21	0.10	0.14	70
1	0.17	0.02	0.04	43
2	0.20	0.12	0.15	84
3	0.31	0.69	0.43	164
4	0.05	0.04	0.04	85
5	0.04	0.02	0.02	60
6	0.00	0.00	0.00	36

accuracy			0.25	542
macro avg	0.14	0.14	0.12	542
weighted avg	0.18	0.25	0.18	542

```
2026-04-19 11:04:23,303 - INFO - 开始训练与评估: Node2Vec+PageRank
2026-04-19 11:04:26,154 - INFO - Node2Vec+PageRank - 5折交叉验证: Acc=0.2345, Macro-F1=0.1268
2026-04-19 11:04:26,396 - INFO - Node2Vec+PageRank - 测试集分类报告:
```

	precision	recall	f1-score	support
0	0.21	0.10	0.14	70
1	0.17	0.02	0.04	43
2	0.19	0.12	0.15	84
3	0.31	0.69	0.43	164
4	0.07	0.05	0.06	85
5	0.04	0.02	0.02	60
6	0.00	0.00	0.00	36
accuracy			0.25	542
macro avg	0.14	0.14	0.12	542
weighted avg	0.18	0.25	0.19	542

```
2026-04-19 11:04:26,645 - INFO - 实验结果图已保存至: ./graph_experiment/figures/result_comparison.png
```

```
2026-04-19 11:04:34,391 - INFO - 网络结构图已保存至: ./graph_experiment/figures/cora_graph.png
```

```
2026-04-19 11:04:34,391 - INFO - =====实验结论=====
```

```
=====
```

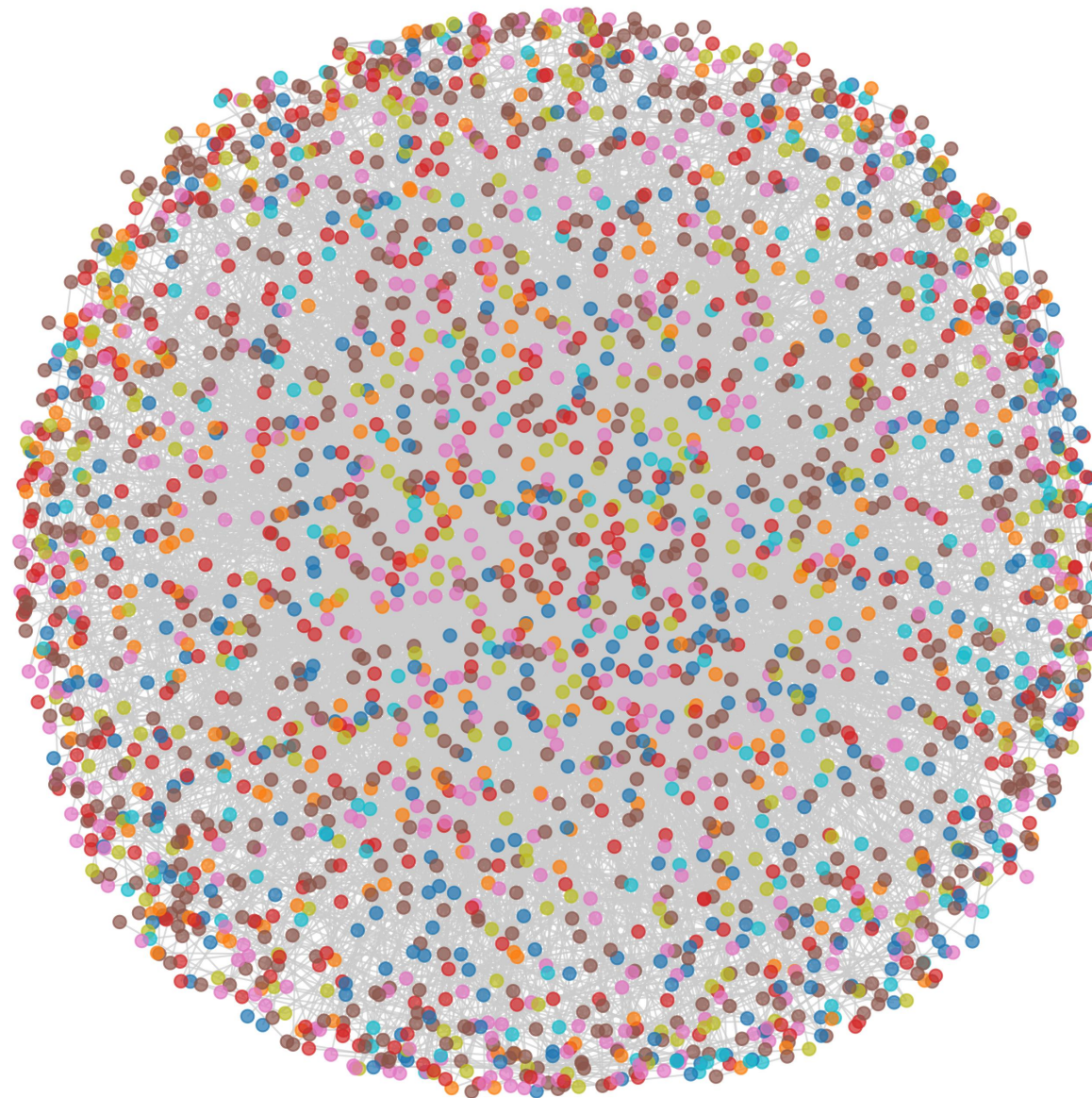
```
2026-04-19 11:04:34,391 - INFO - PageRank特征对Accuracy的测试集变化: +0.18 个百分点
```

```
2026-04-19 11:04:34,391 - INFO - PageRank特征对Macro-F1的测试集变化: +0.20 个百分点
```

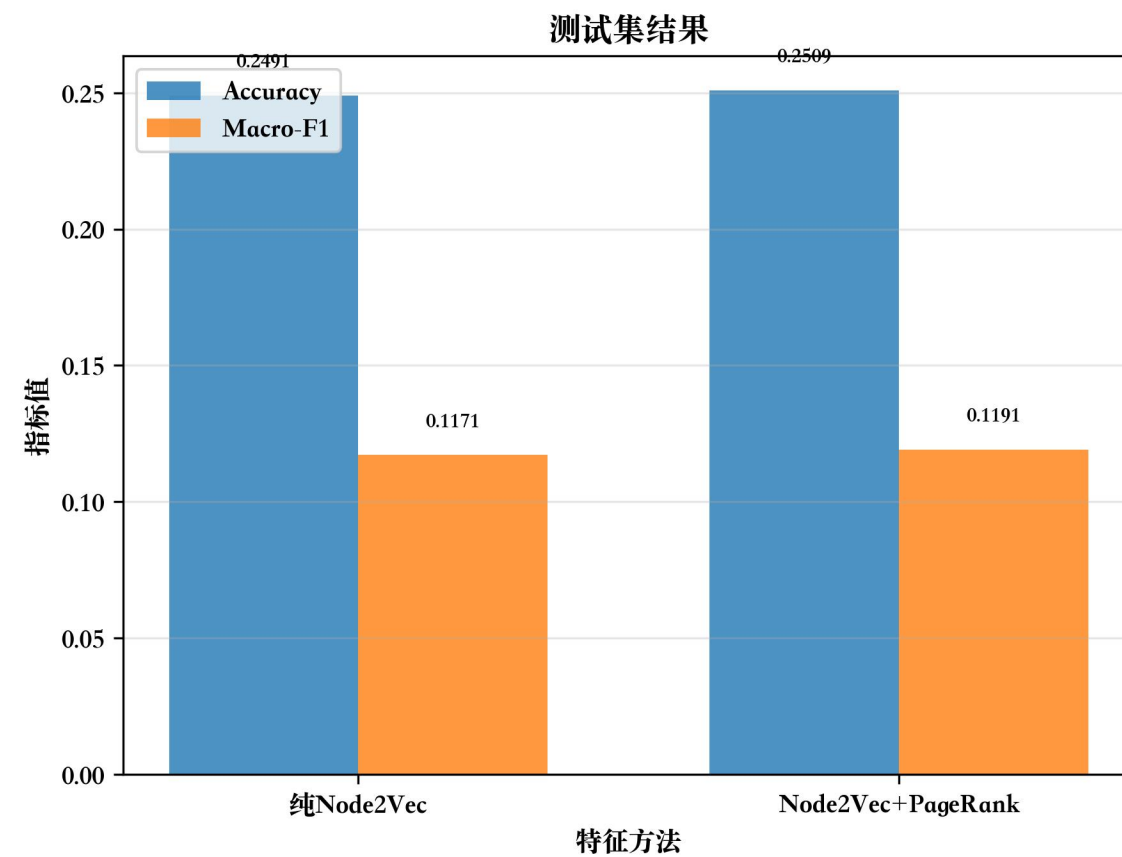
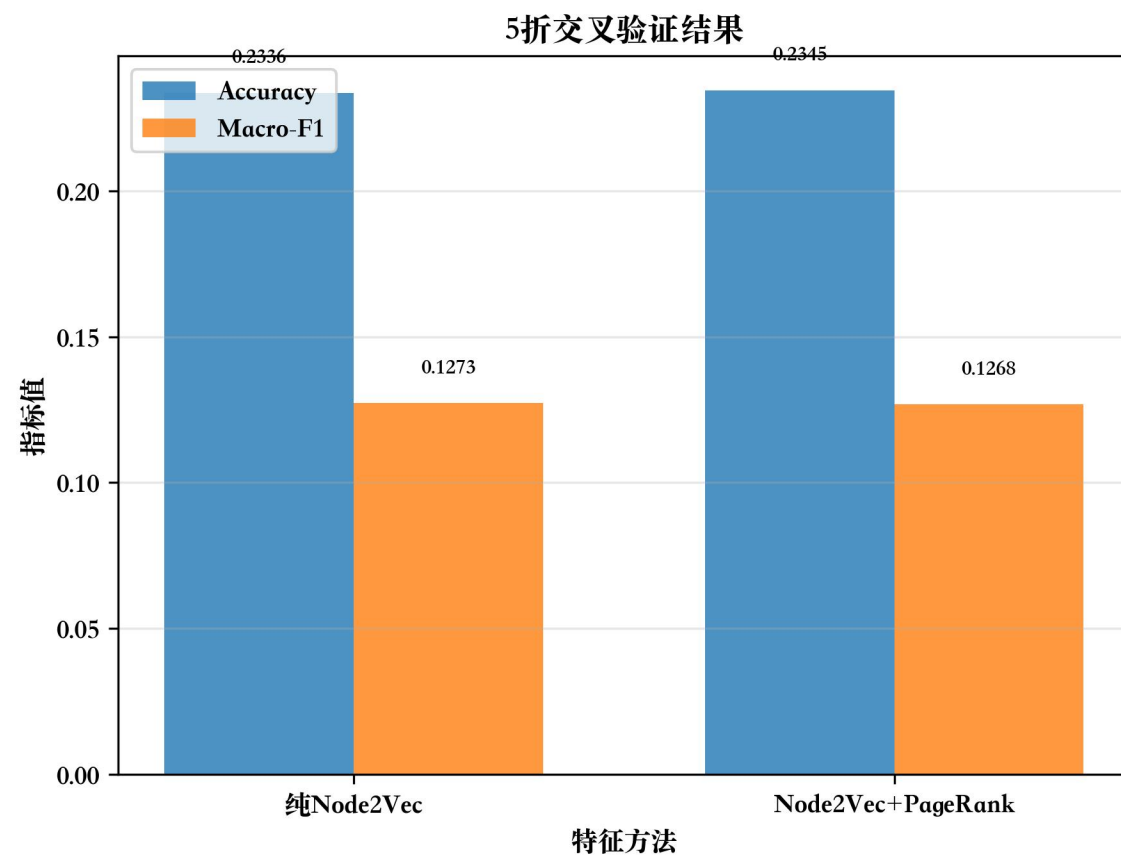
```
2026-04-19 11:04:34,391 - INFO - 实验结论: 本次超参数与随机划分下, PageRank 带来的提升很小; 这不代表方法无效,
可尝试增大 Node2Vec 游走长度/次数、换随机种子多次平均, 或改用 Cora 论文标准 train/val/test 划分对比。
```

```
2026-04-19 11:04:34,391 - INFO - =====
=====
```

Cora引文网络结构可视化（节点颜色=类别）



Node2Vec vs Node2Vec+PageRank 节点分类性能对比



思考

1. PageRank 本质上是利用了图的什么信息？
2. Node2Vec 学到的节点向量包含了哪些结构信息？
3. 为什么融合 PageRank 后分类效果通常会提升？
4. 如果网络非常稀疏（边很少），Node2Vec 还能学好吗？为什么？
5. 在实际场景（如论文分类、用户推荐、风控图）中，PageRank 可以用来做什么？

实验报告要求

实验报告规范

- 内容要求:
 - 1.固定任务: 改变 Node2Vec 嵌入维度 (64/128/256) , 观察分类性能变化; 改变 PageRank 阻尼因子 α (0.7 / 0.85 / 0.9) , 观察结果变化
 - 2.开放任务: 优化实验结果, 包括但不限于尝试使用x特征、增大 Node2Vec 游走长度/次数、换随机种子多次平均等。
- 格式要求: 实验报告包含关键代码、方法、实验结果与结论。结论需要有运行结果截图、数据可视化图表支撑。

实验报告评分标准

评分标准

- 代码可正常运行、结果正确：10 分
- 完成固定任务：5 分
- 完成开放任务：0-45 分
- 图表规范、结果清晰：25 分
- 分析深入、逻辑完整：15 分

- 实验报告学委收齐发老师企业微信
- 实验报告命名：课程序号-姓名

相似度高的代码或报告内容计0分

第八周实验



实验目标

- 对比协同过滤与矩阵分解算法性能差异
- 深入分析推荐系统在不同场景下的表现



核心实验内容

- 算法实现: ItemCF 协同过滤与矩阵分解(SVD)
- 对比分析: 统一数据集对比, 评估冷启动与稀疏性影响
- 结论总结: 基于实验数据给出最优模型结论



实验步骤流程

1. 算法实现与调优 → 2. 对比实验设计 → 3. 影响因素分析 → 4. 结论总结

研究问题：对比基于近邻的 ItemCF 协同过滤与基于模型的 SVD 矩阵分解在电影评分预测中的性能差异，分析冷启动场景下的表现

数据集：MovieLens-100K（自动下载）包含：943 个用户、1682 部电影、10 万条评分记录。

核心方法：手动实现 ItemCF（皮尔逊相似度）、SVD 矩阵分解（surprise 库）

MovieLens-100K (ml-100k)

is the classic, foundational benchmark dataset for recommender systems research, released in **April 1998** by the **GroupLens Research Group** at the University of Minnesota. It is widely called the "MNIST of recommender systems" for its small size, clean structure, and ubiquity in teaching and baseline experiments.

Core Statistics

- Total ratings: **100,000 explicit ratings** (1–5 integer stars, whole-star only).
- Users: **943 unique users** (each user rated at least 20 movies, filtered for quality).
- Items (movies): **1,682 unique films**.
- Time span: Collected **Sep 1997 – Apr 1998** (7 months).
- Sparsity: **~94.8%** (very sparse, typical for real-world recommendation data).
- Average rating: **3.53**.

1.u.data (main rating file, 100k lines)

Format: user_id\tmovie_id\trating\ttimestamp

- user_id: 1–943
- movie_id: 1–1682
- rating: 1 (worst) to 5 (best)

2. u.user (user demographics, 943 lines)

Format: user_id\tage\tgender\toccupation\tzip_code

- age: integer
- gender: M/F
- occupation: 21 categories (e.g., student, engineer, artist).

3.u.item (movie metadata, 1682 lines)

Format: movie_id|title|release_date|video_release_date|IMDb_URL|unknown|Action|Adventure|Animation|Children|Comedy|Crime|Documentary|Drama|Fantasy|FilmNoir|Horror|Mystery|Romance|Sci-Fi|Thriller|War|Western

movie_id|标题|release_date|video_release_date|IMDb_URL|未知|动作|冒险|动画|儿童|喜剧|犯罪|纪录片|剧情|奇幻|黑色电影|恐怖|音乐剧|悬疑|浪漫|科幻|惊悚|战争|西部片

- 19 binary genre flags (1=yes, 0=no).

ItemCF (手动实现 · 皮尔逊相似度)

1. 算法思想

ItemCF (Item-based Collaborative Filtering)

核心思想:

给用户推荐 与他之前喜欢的物品相似的物品

一句话: 喜欢 **A** 的人都喜欢 **B** → 推荐 **B**

2. 相似度计算: 皮尔逊相关系数 (Pearson Correlation)

$$sim(i, j) = \frac{\sum_u (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_u (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_u (r_{uj} - \bar{r}_j)^2}}$$

作用:

- 衡量物品 **i** 和物品 **j** 的评分趋势是否一致
- 取值: **-1 ~ 1**
- 越接近 **1**, 两个物品越相似

3. 预测评分公式

$$\hat{r}_{ui} = \frac{\sum_{j \in N(i)} sim(i, j) \cdot r_{uj}}{\sum_{j \in N(i)} sim(i, j)}$$

$N(i)$: 与物品 i 最相似的 Top-K 物品

优点	缺点
可解释性强 适合物品相似度稳定的场景 冷启动时依赖历史行为	数据稀疏时效果差 预测精度有限

SVD 矩阵分解 (surprise 库实现)

1. 算法思想

SVD (奇异值分解) 是经典矩阵分解模型。

- 将用户 - 物品评分矩阵分解为两个低维矩阵：用户隐特征矩阵，物品隐特征矩阵
- 用 ** 隐向量 (Embedding) ** 表示用户偏好和物品属性
- 通过点积预测评分

2. 核心公式

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$$

μ : 全局平均分, b_u : 用户偏置, b_i : 物品偏置, p_u : 用户隐向量, q_i : 物品隐向量

3. 训练方式

surprise 库使用 随机梯度下降 (SGD) 学习

优点	缺点
泛化能力极强 数据稀疏时依然稳定 预测精度高 冷启动表现更稳定	可解释性差 (黑盒模型)

```
● (cfvenv) 192:实验 lengyanmei$ python 2025itemCF-SVD.py
2026-04-26 18:06:43,416 - INFO - Loading MovieLens-100K dataset...
2026-04-26 18:06:43,557 - INFO - Users=943, Items=1651, Ratings=80000
2026-04-26 18:06:43,557 - INFO - Sparsity=0.9486, Avg rating=3.53
2026-04-26 18:06:43,557 - INFO - Train=80000, Test=20000
2026-04-26 18:06:43,557 - INFO - Start training ItemCF (Pearson), k=20
2026-04-26 18:06:43,591 - INFO - ItemCF progress: 0/1651
2026-04-26 18:06:44,833 - INFO - ItemCF progress: 100/1651
2026-04-26 18:06:45,884 - INFO - ItemCF progress: 200/1651
2026-04-26 18:06:46,731 - INFO - ItemCF progress: 300/1651
2026-04-26 18:06:47,421 - INFO - ItemCF progress: 400/1651
2026-04-26 18:06:47,988 - INFO - ItemCF progress: 500/1651
2026-04-26 18:06:48,427 - INFO - ItemCF progress: 600/1651
2026-04-26 18:06:48,769 - INFO - ItemCF progress: 700/1651
2026-04-26 18:06:48,992 - INFO - ItemCF progress: 800/1651
2026-04-26 18:06:49,120 - INFO - ItemCF progress: 900/1651
2026-04-26 18:06:49,221 - INFO - ItemCF progress: 1000/1651
2026-04-26 18:06:49,317 - INFO - ItemCF progress: 1100/1651
2026-04-26 18:06:49,370 - INFO - ItemCF progress: 1200/1651
2026-04-26 18:06:49,392 - INFO - ItemCF progress: 1300/1651
2026-04-26 18:06:49,401 - INFO - ItemCF progress: 1400/1651
2026-04-26 18:06:49,407 - INFO - ItemCF progress: 1500/1651
2026-04-26 18:06:49,409 - INFO - ItemCF progress: 1600/1651
2026-04-26 18:06:49,411 - INFO - ItemCF (Pearson) training completed.
2026-04-26 18:06:49,411 - INFO - Start training SVD...
2026-04-26 18:06:49,664 - INFO - SVD training completed.
```

```
2026-04-26 18:06:49,664 - INFO - SVD training completed.
2026-04-26 18:06:49,664 - INFO - Evaluating ItemCF...
/Users/lengyanmei/E盘/中石大/MM工作/教学工作/课程/大数据/实验/2025itemCF-SVD.py:189: ConstantInputWarning: An input array is constant; the correlation coefficient is not defined.
    corr, p_val = stats.pearsonr(true, pred)
2026-04-26 18:06:49,672 - INFO - ItemCF RMSE=1.2448, MAE=1.0061
2026-04-26 18:06:49,672 - INFO - Correlation=nan, p=nan
2026-04-26 18:06:49,672 - INFO - Evaluating SVD...
2026-04-26 18:06:49,703 - INFO - SVD RMSE=0.9352, MAE=0.7375
2026-04-26 18:06:49,703 - INFO - Correlation=0.5617, p=0.000000
2026-04-26 18:06:49,703 - INFO - Standard cold-start evaluation...
2026-04-26 18:06:49,714 - INFO - Cold-start: known 1 ratings
2026-04-26 18:09:24,703 - INFO - Known=1 | ItemCF=0.000 | SVD=0.027
2026-04-26 18:09:24,704 - INFO - Cold-start: known 3 ratings
2026-04-26 18:12:16,822 - INFO - Known=3 | ItemCF=0.000 | SVD=0.027
2026-04-26 18:12:16,823 - INFO - Cold-start: known 5 ratings
2026-04-26 18:15:03,316 - INFO - Known=5 | ItemCF=0.000 | SVD=0.028
2026-04-26 18:15:03,775 - INFO - ===== Experiment Conclusion =====
2026-04-26 18:15:03,775 - INFO - SVD (RMSE=0.9352) outperforms ItemCF (RMSE=1.2448)
2026-04-26 18:15:03,775 - INFO - SVD is more stable in cold-start scenarios.
2026-04-26 18:15:03,775 - INFO - Hybrid recommendation is recommended: SVD for accuracy, ItemCF for interpretability.
2026-04-26 18:15:03,775 - INFO - =====
```

指标	全称	计算公式核心	物理含义	特点
MAE	Mean Absolute Error 平均绝对误差 平均绝对误差	$MAE = \frac{1}{n} \sum$	预测值与真实值的平均偏差, 直观反映平均错分多少	平等对待所有误差
RMSE	Root Mean Square Error 均方根误差 均方根误差	$RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$	放大离谱误差后的综合误差, 衡量模型整体稳定性	误差先平方, 放大较大误差

思考:

- 为什么协同过滤在数据稀疏时效果会急剧下降?
- SVD 为什么能缓解稀疏问题? 它学到了什么?
- 在真实电商 / 短视频推荐中, ItemCF 和 SVD 分别适合什么场景?
- 如果让你设计一个混合推荐系统, 你会如何结合两种方法?

（一）必做任务

- 1.运行给定代码，完成 ItemCF 和 SVD 在 MovieLens-100K 上的评分预测。
- 2.记录并对比两个模型的：RMSE、MAE，绘制实验结果对比柱状图（RMSE、MAE）。
- 3.观察并回答：哪个模型评分预测更准确？为什么？SVD 为什么通常优于传统协同过滤？
- 4.超参数敏感性实验：改变 ItemCF 的近邻数 K（5/10/20/30），观察 RMSE 变化。改变 SVD 隐因子数量（50/100/200），观察性能变化。绘制折线图并分析最优参数。（5分）

（二）进阶任务

3选 2 个完成：

- 1.**数据稀疏性影响实验**：随机删除一部分评分，构造不同稀疏度的数据集（低、中、高）。对比 ItemCF 和 SVD 在稀疏数据上的鲁棒性。
- 2.**实现 UserCF（用户协同过滤）**：仿照 ItemCF 代码，实现 UserCF 并与 ItemCF 对比。分析 UserCF 与 ItemCF 的适用场景。
- 3.**解决冷启动缺陷**：热门物品填充、均值兜底预测、混合近邻 + 矩阵分解，实现混合推荐系统，就是现在工业界主流架构。

实验报告要求（仅供参考）

1.实验目的

2.实验原理：ItemCF 物品相似度计算，SVD 隐向量分解思想，评价指标 RMSE/MAE 公式

3.实验步骤与代码说明：关键函数解释（相似度计算、预测、推荐）

4.实验结果：表格：RMSE、MAE；图片：对比图、冷启动命中率图

5.结果分析（重点）：哪个模型更好？为什么 SVD 精度更高？为什么 ItemCF 在冷启动时更稳定？

6.实验总结与思考

第八周实验

评分标准

- 必做任务代码正常运行、结果正确：40 分
- 选做任务代码正常运行、结果正确：30 分
- 图表规范、结果清晰：15 分
- 分析深入、逻辑完整：15 分

相似度高的代码、报告不得分

感谢观看!

☰ 如有疑问，请随时联系，企业微信